Wine Quality Prediction

DAT0424 - Achieving a Machine Learning Proof of Concept

Student ML project using the Wine Quality Dataset.

Goal: predict quality_group from wine measurements and wine_type

WHAT THIS POC TRIES TO DO

Project Context and Objective

Business objective

Help a producer, distributor, or laboratory quickly estimate the probable quality group of a wine using physicochemical characteristics.

- Useful as a first screening before more detailed sensory analysis.
- The output is a quality group, not an exact expert score.
- This is a first ML proof of concept, not a production system.

POC scope

Clean data

Feature engineering

Three models

Mock-up dashboard

FILES AND BASIC CHECKS

Dataset Overview

1,599	4,898	6,497	0
red wine rows	white wine rows	total rows after merging	missing values detected

- Source files: winequality-red.csv and winequality-white.csv.
- The data is structured and tabular, with one row per wine sample.
- No time variables were found, so no date-time feature engineering was needed.
- winequality.names documents the variables and dataset context.

WHAT THE MODEL SEES

Inputs and Target

Input features

- fixed acidity
- volatile acidity
- citric acid
- residual sugar
- chlorides
- free sulfur dioxide
- total sulfur dioxide
- density
- pH
- sulphates
- alcohol
- wine_type

Target

quality_group

Classes: low quality, medium quality, high quality.

quality is not used as an input feature to avoid leakage.

FROM QUALITY SCORE TO CLASSIFICATION GROUPS

Target Engineering

Original target: quality

Engineered target: quality_group

low quality	quality <= 5
medium quality	quality == 6
high quality	quality >= 7

Why classification?

The original score is useful, but grouping it into three classes makes the first POC easier to explain and defend in a student presentation.

HOW THE GRAPHS SUPPORT THE STORY

Role of the Visualizations

Original quality distribution

Shows how the raw quality scores are spread before target engineering.

Quality group distribution

Shows the final classes used for classification and their imbalance.

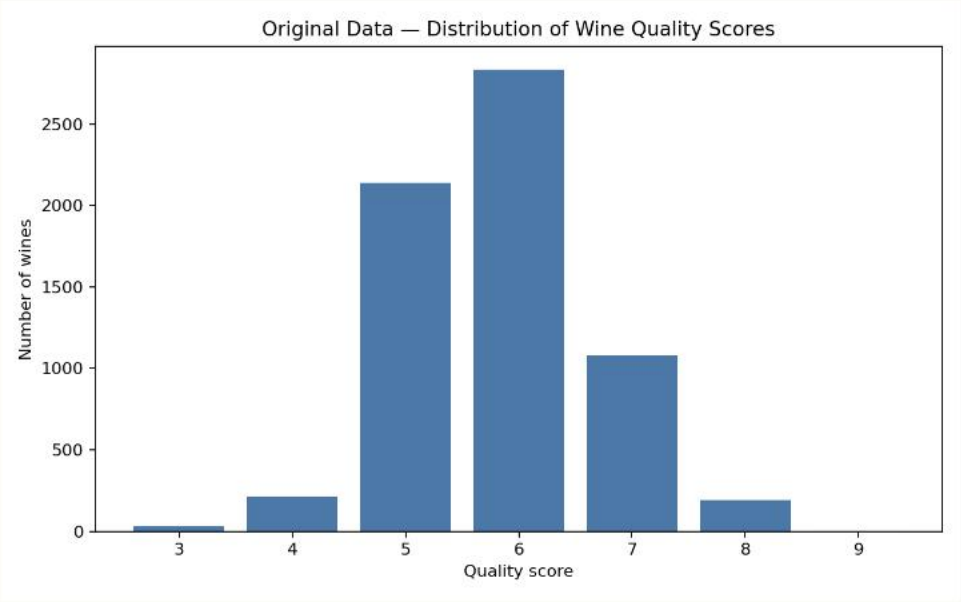
Model performance comparison

Shows which of the three models performs best using macro F1-score.

Together, these visuals explain the data, the engineered target, and the model choice without adding unnecessary graphs.

RAW DATA BEFORE TARGET ENGINEERING

Graph 1 - Original Quality Distribution



What it shows

Most wines are around middle quality scores. Very low and very high scores are rare.

Why it matters

It helps justify simplifying the task into three quality groups.

CLASS DISTRIBUTION AFTER FEATURE ENGINEERING

Graph 2 - Final Classification Target

What it shows

The final classes low, medium, and high are not perfectly balanced.

Why it matters

This supports using macro F1-score instead of relying only on accuracy.

SIMPLE PREPROCESSING WITH LEAKAGE PREVENTION

Feature Engineering Pipeline

01
Add wine_type

02
Merge red and white

03
Create quality_group

04
Split train/test

05
Median imputation

06
Standardization

07
One-hot wine_type

08
Remove target columns

The train/test split is done before preprocessing, and preprocessing is fitted only on the training data.

KEEPING THE MODEL COMPARISON FAIR

Evaluation Protocol

Same setup for all models

- Same engineered dataset.
- Same train/test split.
- Same target: quality_group.
- Same final test set and same metrics.

Leakage prevention

- quality is removed from input features.
- quality_group is the target only.
- Preprocessing is fitted only on X_train.

THREE SIMPLE MODEL FAMILIES

Models Compared

Logistic Regression

Simple baseline

Fast, understandable, and useful as a reference model.

Random Forest

Flexible non-linear model

Can capture interactions between physicochemical variables.

SVM Classifier

Margin-based model

Uses standardized features and gives a different comparison logic.

No heavy tuning was used because the goal is a first student proof of concept.

Metrics Used

Main metric

macro F1-score

Macro F1 gives each class more balanced importance, which is useful because the classes are not perfectly balanced.

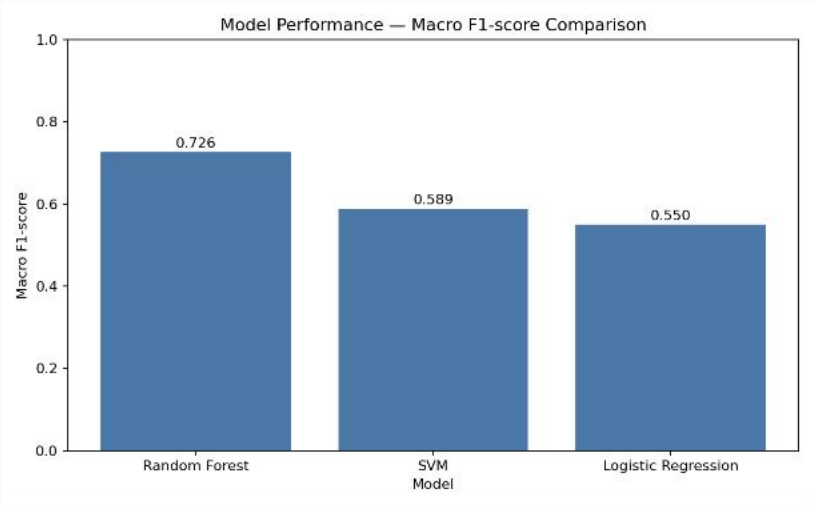
Secondary metrics

- accuracy
- precision_macro
- recall_macro

Accuracy is still useful, but it can hide weak results on minority classes.

FIRST POC COMPARISON

Model Results and Performance Graph



Model	Acc.	Prec.	Rec.	F1
Logistic Regression	0.582	0.585	0.539	0.550
Random Forest	**0.732**	**0.748**	**0.712**	**0.726**
SVM	0.627	0.637	0.576	0.589

Random Forest performed best in this first comparison with macro F1 = 0.726.

This is not a final production model.

FUTURE INTERFACE IDEA

Streamlit Mock-up and Dashboard Journey

User journey

1. Open the app -> 2. Read the objective -> 3. Enter wine characteristics -> 4. See a mocked prediction -> 5. View model story and performance -> 6. Future version connects the trained model.

WHAT REMAINS AFTER THIS POC

Limitations, Next Steps, and Conclusion

Limitations

- Model is not final.
- Tuning is limited.
- Wine quality is partly subjective.
- No region, price, grape variety, vintage, or brand.
- Streamlit is not connected to a real backend yet.

Next steps

- Connect final preprocessing.
- Load the selected trained model.
- Replace mocked prediction with real prediction.
- Test the app with new examples.
- Improve validation and possibly tune the best model.

Conclusion

This project built a complete student ML POC: data preparation, visual analysis, three-model comparison, and a Streamlit mock-up.

GitHub repository link: [to be added by the student]